Academic Documentation: react-native-pose-landmarks

1. Package Overview (The "What")

1.1 Primary Responsibility

`react-native-pose-landmarks` is a React Native Nitro module that provides real-time human pose landmark detection by interfacing with Google's MediaPipe Pose Landmarker library. The package captures video frames from the device front camera and processes them through a machine learning inference pipeline to extract 33 distinct anatomical landmark points (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles, etc.) with associated spatial coordinates and visibility confidence scores.

1.2 Problem Domain

Within the broader "workout-hacker" system, this package solves the fundamental computer vision problem of real-time human pose estimation on mobile devices. It serves as the foundational sensing layer that transforms raw camera input into structured skeletal data that downstream packages (such as `react-native-exercise-recognition` and `react-native-rep-counter`) consume to perform exercise classification and repetition counting.

The package addresses several technical challenges specific to mobile deployment: - **Resource constraints**: Efficient inference on mobile hardware without excessive battery drain - **Real-time processing**: Achieving near-30 FPS landmark extraction with minimal latency - **Noisy input handling**: Managing partial occlusions, poor lighting, and camera motion - **Cross-platform abstraction**: Providing a unified JavaScript API across Android and iOS

1.3 Package Position in System Architecture

The package occupies the foundational sensing layer in the workout-hacker system:

1. Camera Hardware captures video frames
2. react-native-pose-landmarks extracts skeletal landmarks
3. react-native-exercise-recognition classifies exercise type from landmarks
4. react-native-rep-counter counts repetitions

2. Architectural Rationale (The "Why")

2.1 Technology Selection: Nitro Modules

The package was constructed using the React Native Nitro Modules framework. This choice was made to achieve near-native performance for the computationally intensive image processing and inference operations. Traditional React Native

bridge architecture incurs significant overhead for high-frequency data transfers, particularly problematic when handling 30 FPS video streams. Nitro's hybrid object architecture enables direct memory access and efficient serialization, critical for real-time computer vision workloads.

The specification in `src/specs/pose-landmarks.nitro.ts` defines the interface contract between JavaScript and native code:

```
export interface PoseLandmarks extends HybridObject<{ ios: 'swift', android: 'kotlin' }> {
  initPoseLandmarker(...): boolean;
  closePoseLandmarker(): boolean;
  getLandmarksBuffer(): Array<number>;
  getLastInferenceTimeMs(): number;
}
```

2.2 MediaPipe Adoption

Google's MediaPipe Pose Landmarker was selected as the underlying machine learning engine. This decision reflects several engineering considerations:

1. **Model variety**: MediaPipe offers three model variants (Lite, Full, Heavy) allowing deployment to trade accuracy for speed based on device capability
2. **Cross-platform consistency**: Single model definition produces consistent results across Android and iOS
3. **Live streaming mode**: Native support for streaming inference with result callbacks rather than batch processing
4. **No server dependency**: On-device inference ensures privacy and eliminates network latency

2.3 Default Configuration Decisions

The package defaults were carefully selected based on empirical testing:

Parameter	Default Value	Rationale
Model	Lite (1)	Balances speed and accuracy; Full model introduces latency
Delegate	CPU (0)	GPU delegate requires specific driver support; CPU ensures compatibility
Inference Rate	30 Hz	Matches vision based ML models sample rate requirements; higher rates incur battery cost

Parameter	Default Value	Rationale
Visibility Threshold	0.9	High threshold filters noisy detections while maintaining responsiveness
One Euro Filter	Enabled	Essential for smoothing landmark jitter; critical for other ML models accuracy

These defaults represent the "safe path" prioritizing startup reliability and stability over maximum performance. The API exposes configuration options for developers to tune parameters when they have specific performance requirements.

2.4 Post-Processing Pipeline Architecture

The package implements a multi-stage post-processing pipeline that refines raw MediaPipe output before exposing it to JavaScript consumers:

1. **Visibility Recovery**: When landmark confidence drops below threshold, the landmark position is frozen to the last known good position rather than jumping to incorrect locations
2. **One Euro Filter**: A variant of the Euro filter algorithm that adapts cutoff frequency based on velocity, effectively smoothing high-frequency jitter while preserving actual movement

Early versions included a rigid body constraint to enforce physical consistency across landmarks. However, testing showed it was freezing too many valid frames—holding onto old positions instead of updating them. The feature was removed to prioritize responsive frame updates over strict physical constraints.

2.5 Trade-offs Considered

Performance vs. Accuracy: The Lite model was chosen as default over Full/Heavy models. While Full produces more accurate landmarks, the latency increase impacts real-time requirements of the broader system. The One Euro filter compensates for Lite's reduced accuracy by smoothing noisy frames.

3. Internal Mechanics

3.1 Initialization Flow

Upon calling `initPoseLandmarker()`, the native Android implementation (`HybridPoseLandmarks.kt`) executes the following sequence:

1. **Configuration application**: Parameters are validated and clamped to valid ranges
2. **Thread pool creation**: Creates dedicated executor services for camera processing and output scheduling
3. **MediaPipe initialization**: Loads the pose landmarker model from app assets
4. **Camera binding**: Attaches an ImageAnalysis use case to CameraX using the front camera
5. **Prediction loop start**: Begins a scheduled task that publishes processed frames at 30Hz

```
initPoseLandmarker()
    |
    v
applyProcessingConfig() -> validation & state initialization
    |
    v
Create ExecutorServices -> single-thread executors for isolation
    |
    v
PoseLandmarkerHelper() -> MediaPipe model loading
    |
    v
ProcessCameraProvider.bindToLifecycle() -> CameraX binding
    |
    v
startPredictionLoop() -> ScheduledExecutorService at 33ms intervals
```

3.2 Camera Frame Processing Pipeline

The core processing occurs in the `ImageAnalysis.Analyzer` callback within `HybridPoseLandmarks.kt:194-206`:

```
Camera Frame (256x256 RGBA)
         |
         v
PoseLandmarkerHelper.detectLiveStream()
         |
         +-- Convert ImageProxy to Bitmap
         |
         +-- Convert Bitmap to MPImage (MediaPipe format)
         |
         +-- Apply rotation from image metadata
         |
         +-- poseLandmarker.detectAsync()
                   |
                   v
```

```
MediaPipe Inference
        |
        v
onResults() callback
        |
        +-- Extract 33 landmarks
        |
        +-- Transform coordinates (Y->X, flip X for front camera)
        |
        +-- processAndStoreFrame()
        |       |
        |       +-- Visibility Recovery
        |       |
        |       +-- One Euro Filtering
        |       |
        |       +-- Store to latestLandmarks buffer
        |
        +-- Return to JS via polling
```

3.3 Post-Processing Algorithm: One Euro Filter

The One Euro Filter implementation (`OneEuroFilter` class, lines 471-521) is based on the algorithm described by Casiez et al. (2012). The key innovation is an adaptive cutoff frequency:

```
val cutoff = minCutoff + beta * abs(smoothedDerivative)
val alpha = alpha(frequency, cutoff)
val smoothedValue = lowPass(alpha, value, previousValue)
```

When landmark velocity is high (during exercise movement), the cutoff increases, allowing the filter to respond quickly. When stationary, the cutoff decreases, heavily smoothing any detection noise. The default parameters (`minCutoff=1.0`, `beta=0.009`) were empirically tuned to minimize standing still jitter while preserving exercise movement.

3.4 Output Buffer Format

The `getLandmarksBuffer()` method returns a flattened `DoubleArray` of 132 elements (33 landmarks $\times$ 4 values):

Index Range	Field	Description
0, 4, 8, ...	x	Normalized X coordinate (0..1, left-to-right)
1, 5, 9, ...	y	Normalized Y coordinate (0..1, top-to-bottom)
2, 6, 10, ...	z	Depth (normalized, relative to hip depth)
3, 7, 11, ...	visibility	Confidence score (0..1)

The normalization to 0..1 simplifies downstream consumption, allowing UI rendering to scale to any canvas size without coordinate transformation.

3.5 Resource Management

The `closePoseLandmarker()` method performs comprehensive cleanup: - Unbinds CameraX use cases on the main thread - Shuts down executor services - Clears MediaPipe landmarker resources - Resets state variables

This ensures no resource leaks when the JavaScript consumer unmounts or re-initializes.

4. Interface & Data Flow (Inputs and Outputs)

4.1 Inputs

Input Source	Type	Description
Front Camera	CameraX ImageAnalysis	256×256 RGBA frames at up to 30 FPS
Configuration	Function parameters	Initialization-time tuning parameters

Initialization Parameters (`initPoseLandmarker()`):

```
interface InitConfig {
  minVisibilityConfidence?: number   // 0.0-1.0, default 0.9
  inferenceSampleRateHz?: number     // 1-30, default 30
  modelSelection?: number            // 0=Full, 1=Lite, 2=Heavy, default 1
  enableVisibilityRecovery?: boolean // default true
  enableRigidBodyConstraint?: boolean // deprecated, ignored
  enableOneEuroFilter?: boolean     // default true
  enableMotionPrediction?: boolean  // default false
  oneEuroMinCutoff?: number          // 0.01-5.0, default 1.0
  oneEuroBeta?: number               // 0.0-1.0, default 0.009
}
```

The camera frames originate from the device's front-facing camera via CameraX's `ImageAnalysis` use case. The package hardcodes usage of the front camera (`CameraSelector.DEFAULT_FRONT_CAMERA`) to ensure consistent landmark orientation for exercise tracking applications.

4.2 Outputs

Output	Type	Description
Landmark Buffer	`Array<number>`	132-element flattened array (33×4)

Output	Type	Description
Inference Time	`number`	Milliseconds for last MediaPipe inference
Initialization Status	`boolean`	Success/failure of initPoseLandmarker()

Landmark Data Structure:

```
Buffer Layout (indices):
+-------------------------------------------------------------------+
| Landmark 0 (nose)                                                 |
| [x0, y0, z0, visibility0]                                         |
+-------------------------------------------------------------------+
| Landmark 1 (left_eye_inner)                                       |
| [x1, y1, z1, visibility1]                                        |
+-------------------------------------------------------------------+
| ...                                                               |
+-------------------------------------------------------------------+
| Landmark 32 (left_foot_index)                                     |
| [x32, y32, z32, visibility32]                                     |
+-------------------------------------------------------------------+
```

4.3 Data Destination

The landmark buffer is consumed by downstream packages in the workout-hacker system:

1. **`react-native-exercise-recognition`**: Processes sequential landmark frames to classify exercise type.
2. **`react-native-rep-counter`**: Monitors landmark positions to detect exercise repetition cycles.

5. Integration & Usage

5.1 Direct Usage Example

The following example demonstrates direct consumption of the pose landmarks package within a React Native component:

```
import React, { useEffect, useState } from 'react'
import { PoseLandmarks } from 'react-native-pose-landmarks'

const LANDMARK_COUNT = 33
const VALUES_PER_LANDMARK = 4

export function usePoseLandmarks() {
```

```
  const [landmarks, setLandmarks] = useState<number[]>([])
  const [inferenceMs, setInferenceMs] = useState<number>(-1)

  useEffect(() => {
    // Initialize the native pose landmarker
    const initialized = PoseLandmarks.initPoseLandmarker(
      0.95,  // minVisibilityConfidence
      30,    // inferenceSampleRateHz
      undefined, // rigidBodyWindowFrames (deprecated)
      1,     // modelSelection (1 = Lite)
      true,  // enableVisibilityRecovery
      true,  // enableOneEuroFilter
    )

    if (!initialized) {
      console.error('Failed to initialize pose landmarker')
      return
    }

    // Poll landmark buffer at 30Hz
    const interval = setInterval(() => {
      const buffer = PoseLandmarks.getLandmarksBuffer()
      // Verify full frame received (33 landmarks × 4 values = 132)
      if (buffer.length === LANDMARK_COUNT * VALUES_PER_LANDMARK) {
        setLandmarks(buffer)
      }
      setInferenceMs(PoseLandmarks.getLastInferenceTimeMs())
    }, 33) // ~30 FPS

    return () => {
      clearInterval(interval)
      PoseLandmarks.closePoseLandmarker()
    }
  }, [])

  return { landmarks, inferenceMs }
}
```

Documentation generated for academic thesis submission. Package version: 1.2.0